

OpenGL

Computer Graphics — Complete Study Guide

Viewing · Projection · Transformations · Lighting · Colour

Clipping · Coordinates · Event Handling · Textures · Anti-Aliasing

Exam preparation: functions, parameters, and their visual effects

1. Viewing & Projection

gluPerspective — perspective frustum

Sets up a pyramid-shaped (frustum) view volume. This is the most commonly tested function. All geometry outside the frustum is clipped before rasterisation.

```
gluPerspective(fovy, aspect, zNear, zFar);  
// Example:  
gluPerspective(45.0, (float)width/height, 0.1, 100.0);
```

fovy	Vertical field of view in degrees. Small value = telephoto (zoomed in). Large value = wide-angle / fisheye distortion.
aspect	Width / height of the viewport. Must match glViewport dimensions to avoid stretching.
zNear	Distance to near clip plane. Must be > 0. Setting close to 0 destroys depth-buffer precision and causes z-fighting.
zFar	Distance to far clip plane. Objects beyond this are invisible. High far/near ratio degrades z-buffer precision.

Exam tip: Never set $zNear = 0$. Depth precision = $\log(zFar/zNear)$. A near plane of 0.001 and far of 1000 gives ratio 1,000,000 — severe z-fighting.

gluLookAt — camera placement

Defines where the camera is, what it looks at, and which direction is "up". This function computes a rotation+translation matrix that moves the world into camera space.

```
gluLookAt(eyeX,eyeY,eyeZ, centerX,centerY,centerZ, upX,upY,upZ);  
// Camera at (0,2,5) looking at origin, Y is up:  
gluLookAt(0, 2, 5, 0, 0, 0, 0, 1, 0);
```

eye XYZ	Camera position in world space. Moving Z further = objects appear smaller.
center XYZ	The point the camera looks toward. Changing this pans the view.
up XYZ	Defines the "up" direction. (0,1,0) is standard. (0,-1,0) flips the image upside-down.

Exam tip: gluLookAt must be called after glMatrixMode(GL_MODELVIEW) and glLoadIdentity(). It multiplies onto the current modelview matrix.

glOrtho — orthographic projection

Parallel projection — objects keep the same size regardless of depth. Used for CAD drawings, 2D HUDs, isometric views, and shadow maps.

```
glOrtho(left, right, bottom, top, nearVal, farVal);  
// Example – 10x10 unit ortho window:  
glOrtho(-5, 5, -5, 5, 0.1, 100.0);
```

left / right	Horizontal bounds of the view volume. Narrowing these zooms in; widening zooms out.
bottom / top	Vertical bounds. Swapping bottom and top flips the Y axis.
nearVal/farVal	Depth range. Unlike perspective, the near value CAN be negative (behind the camera).

Exam tip: Key exam distinction: glOrtho has no foreshortening. gluPerspective makes distant objects smaller. Use glOrtho for technical drawings, gluPerspective for 3D scenes.

2. Transformations & Modelling

Matrix modes and the matrix stack

OpenGL maintains separate matrix stacks. Always set the correct mode before operating.

```
glMatrixMode(GL_PROJECTION); // switch to projection stack
glLoadIdentity();
gluPerspective(...);

glMatrixMode(GL_MODELVIEW); // switch back for object transforms
glLoadIdentity();
gluLookAt(...);
```

Exam tip: Forgetting `glMatrixMode(GL_MODELVIEW)` before object transforms corrupts the projection matrix — a very common exam bug.

glTranslatef / glRotatef / glScalef

All three multiply onto the current matrix. CRITICAL: OpenGL applies them in reverse code order — the last transform written executes first in object space.

```
glLoadIdentity();
glTranslatef(tx, ty, tz); // move
glRotatef(angle, x, y, z); // rotate around axis (x,y,z)
glScalef(sx, sy, sz); // scale
drawObject(); // object sees: Scale first, then Rotate, then Translate
```

glTranslatef(tx,ty,tz)	Moves origin by (tx,ty,tz). All subsequent draws are offset.
glRotatef(a,x,y,z)	Rotates by angle a (degrees) around axis vector (x,y,z). Normalise the axis.
glScalef(sx,sy,sz)	Scales along each axis. Negative values mirror the object. Non-uniform scale distorts normals.

Exam tip: Order matters: T*R*S applied in code = object is first scaled, then rotated, then translated. Swap the order and you get a different result.

glPushMatrix / glPopMatrix — scene graphs

Saves (push) and restores (pop) the current matrix onto a stack. Essential for hierarchical models where child transforms should not affect siblings.

```
glPushMatrix(); // save shoulder state
glTranslatef(0, armLen, 0);
glRotatef(elbowAngle, 0,0,1);
drawForearm();

glPushMatrix(); // save elbow state
glTranslatef(0, foreLen, 0);
glRotatef(wristAngle, 0,0,1);
drawHand();
glPopMatrix(); // restore elbow state
glPopMatrix(); // restore shoulder state
drawTorso(); // unaffected by arm transforms
```

Exam tip: Stack depth is typically 32. Mismatched push/pop count is a classic bug — transforms accumulate across frames causing drift.

3. Lighting & Shading

Enabling lighting

```
glEnable(GL_LIGHTING); // global lighting switch
glEnable(GL_LIGHT0); // enable light 0 (up to GL_LIGHT7)
glEnable(GL_NORMALIZE); // re-normalise normals after scaling
```

glLightfv — light properties

```
GLfloat pos[] = {1.0, 2.0, 3.0, 1.0}; // w=1: point light
GLfloat diffuse[] = {1.0, 1.0, 1.0, 1.0}; // white diffuse
GLfloat ambient[] = {0.2, 0.2, 0.2, 1.0};

glLightfv(GL_LIGHT0, GL_POSITION, pos);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
```

GL_POSITION w=1	Point light — has a position, attenuates with distance. Highlight moves as light moves.
GL_POSITION w=0	Directional light — infinitely far (like sun). XYZ is the direction vector. No attenuation.
GL_DIFFUSE	RGBA colour of the diffuse (Lambertian) component. Most visible colour on a surface.
GL_SPECULAR	Colour of the specular highlight. Often white for metals.
GL_AMBIENT	Low-level fill light simulating indirect illumination.

Exam tip: w=0 means direction, not position. Light at {0,1,0,0} shines straight down from above infinity.

glMaterialfv — surface material

```
GLfloat mat_diff[] = {0.2, 0.5, 0.8, 1.0}; // blue diffuse
GLfloat mat_spec[] = {1.0, 1.0, 1.0, 1.0}; // white specular

glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diff);
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_spec);
glMaterialf (GL_FRONT, GL_SHININESS, 64.0f); // 1.0 - 128.0
```

GL_DIFFUSE	Main surface colour under white light.
GL_AMBIENT	Colour under ambient light. Usually similar to diffuse.
GL_SPECULAR	Colour of the specular highlight spot.

GL_SHININESS	Phong exponent: 1=very broad matte, 128=tiny mirror-like spot. Maximum is 128 in fixed-function OpenGL.
GL_EMISSION	Surface emits this colour regardless of lighting — makes objects appear self-lit.

glShadeModel — shading algorithm

```
glShadeModel(GL_FLAT); // one colour per polygon (fast, faceted look)
glShadeModel(GL_SMOOTH); // Gouraud: interpolate vertex colours across face
// True Phong shading requires a GLSL fragment shader
```

GL_FLAT	Uses the colour of the last vertex of each primitive. Produces a faceted, hard-edged appearance.
GL_SMOOTH	Gouraud shading — colours computed at each vertex and linearly interpolated across the polygon.
Phong	NOT a glShadeModel option. Requires per-fragment normal interpolation in a GLSL fragment shader.

Exam tip: GL_SMOOTH is Gouraud, not Phong. Gouraud can miss highlights in polygon centres; Phong fixes this but requires shaders.

4. Colour & Blending

RGB colour model & glColor

OpenGL uses normalised float components [0.0, 1.0]. The RGB cube has black at (0,0,0) and white at (1,1,1).

```
glColor3f(r, g, b); // no alpha
glColor4f(r, g, b, a); // with alpha

// Common colours:
glColor3f(1, 0, 0); // red
glColor3f(0, 1, 0); // green
glColor3f(1, 1, 0); // yellow
glColor3f(1, 1, 1); // white
glColor3f(0, 0, 0); // black
```

glBlendFunc — alpha blending

Controls how incoming (src) fragments combine with existing (dst) framebuffer pixels. Must enable GL_BLEND first.

```
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
// Result = src_colour * src_alpha + dst_colour * (1 - src_alpha)
// Additive blending (fire/glow effects):
glBlendFunc(GL_SRC_ALPHA, GL_ONE);
```

GL_SRC_ALPHA	Multiply source (incoming) colour by its own alpha.
GL_ONE_MINUS_SRC_ALPHA	Multiply destination by (1 - src_alpha). Standard transparency.
GL_ONE	Use destination unchanged. Combined with SRC_ALPHA = additive blend.
GL_ZERO	Zero out that factor — used to replace rather than blend.

Exam tip: Draw opaque objects FIRST, then transparent objects sorted back-to-front. Wrong draw order produces incorrect transparency.

5. Clipping & Depth Buffer

Depth buffer (z-buffer)

```
glEnable(GL_DEPTH_TEST);  
glDepthFunc(GL_LESS); // default: pass if incoming z < stored z  
  
// Must clear depth buffer each frame:  
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

GL_LESS	Default. Fragment passes if its depth is closer than stored value.
GL_LEQUAL	Pass if closer or equal. Used for decals drawn at same depth.
GL_ALWAYS	Always pass — effectively disables depth testing.
GL_NEVER	Always fail — nothing is drawn.

Exam tip: Z-fighting: two surfaces at nearly the same depth flicker because they share depth-buffer precision. Fix by increasing near, decreasing far, or using polygon offset.

glClipPlane — user-defined clipping

Defines an arbitrary half-space: the region $ax+by+cz+d \geq 0$ is kept. Up to 6 clip planes (GL_CLIP_PLANE0 through GL_CLIP_PLANE5).

```
GLdouble eq[] = {0, 1, 0, 0}; // keep y >= 0 (upper half)  
glClipPlane(GL_CLIP_PLANE0, eq);  
glEnable(GL_CLIP_PLANE0);  
  
// eq = {a, b, c, d} -> plane: ax + by + cz + d >= 0 survives
```

6. Coordinate Systems & Maths

The OpenGL transformation pipeline

```
// Vertex journey through 4 spaces:  
1. Object space (local model coordinates)  
| glTranslate / glRotate / glScale (Model matrix)  
v  
2. World space  
| gluLookAt (View matrix)  
v  
3. Eye/Camera space  
| gluPerspective / glOrtho (Projection matrix)  
v  
4. Clip space / NDC [-1,1]^3  
| glViewport(x, y, w, h)  
v  
5. Window/Screen pixels
```

Exam tip: In legacy OpenGL, GL_MODELVIEW combines the Model and View matrices. Switch with glMatrixMode(GL_PROJECTION) only for the projection matrix.

Homogeneous coordinates

All transforms are 4x4 matrices operating on (x, y, z, w). Points have w=1; direction vectors have w=0 (unaffected by translation).

```
// Translation matrix T(tx, ty, tz):  
[ 1 0 0 tx ]  
[ 0 1 0 ty ]  
[ 0 0 1 tz ]  
[ 0 0 0 1 ]  
  
// Final: gl_Position = Projection x View x Model x vertex
```

7. Event Handling (GLUT/FreeGLUT)

GLUT uses a callback registration model — you register functions that the windowing system calls when events occur. Your program never polls; it reacts.

Core callback functions

```
// Register callbacks at startup:
glutDisplayFunc(display); // window needs redrawing
glutReshapeFunc(reshape); // window resized
glutKeyboardFunc(keyboard); // ASCII key pressed
glutSpecialFunc(specialKeys); // arrow / function keys
glutMouseFunc(mouseButton); // mouse button press/release
glutMotionFunc(mouseMove); // mouse moved with button held
glutPassiveMotionFunc(passiveMove); // mouse moved, no button
glutIdleFunc(idle); // called when no events pending
glutTimerFunc(ms, timer, value); // one-shot timer callback
```

Keyboard callbacks

```
void keyboard(unsigned char key, int x, int y) {
    switch(key) {
        case 27: exit(0); break; // ESC key
        case 'r': rotY += 5; break;
        case 'R': rotY -= 5; break;
    }
    glutPostRedisplay(); // request redraw
}

// Special keys (arrows, F1-F12, etc.):
void specialKeys(int key, int x, int y) {
    if (key == GLUT_KEY_UP) camZ -= 0.5f;
    if (key == GLUT_KEY_DOWN) camZ += 0.5f;
    if (key == GLUT_KEY_LEFT) camX -= 0.5f;
    if (key == GLUT_KEY_RIGHT) camX += 0.5f;
    glutPostRedisplay();
}
```

key	ASCII value of pressed key. ESC = 27, SPACE = 32, ENTER = 13.
x, y	Mouse cursor position at the time of the key press (window coordinates).
GLUT_KEY_*	Constants for special keys: UP, DOWN, LEFT, RIGHT, F1–F12, HOME, END, PAGE_UP, PAGE_DOWN.

Mouse callbacks

```
void mouseButton(int button, int state, int x, int y) {  
    // button: GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON, GLUT_RIGHT_BUTTON  
    // state: GLUT_DOWN, GLUT_UP  
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {  
        dragging = 1;  
        lastX = x; lastY = y;  
    }  
    if (state == GLUT_UP) dragging = 0;  
}  
  
void mouseMove(int x, int y) { // called when button held  
    if (dragging) {  
        rotY += (x - lastX) * 0.5f;  
        rotX += (y - lastY) * 0.5f;  
        lastX = x; lastY = y;  
        glutPostRedisplay();  
    }  
}
```

Exam tip: `glutPostRedisplay()` marks the window as needing a redraw — it does NOT redraw immediately. The display callback fires on the next event loop iteration.

Window reshape & viewport

```
void reshape(int w, int h) {  
    glViewport(0, 0, w, h); // set pixel drawing area  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    gluPerspective(45.0, (float)w/h, 0.1, 100.0);  
    glMatrixMode(GL_MODELVIEW);  
}
```

glViewport(x,y,w,h)

Maps NDC coordinates to a rectangle of pixels. x,y = lower-left corner; w,h = dimensions.

Exam tip: Always update both `glViewport` AND `gluPerspective` aspect ratio in `reshape()`. Updating only one causes stretching.

glutTimerFunc — software timers

```
void timer(int value) {  
    angle += 2.0f;  
    glutPostRedisplay();  
    glutTimerFunc(16, timer, 0); // re-register for ~60 fps  
}  
  
// In main():  
glutTimerFunc(16, timer, 0); // fire after 16 ms
```

Exam tip: glutIdleFunc fires as fast as possible (burns CPU). glutTimerFunc(16,...) gives ~60 fps with controlled timing. Prefer timer for animations.

8. Texture Mapping

Texture mapping wraps a 2D image onto a 3D surface. Texture coordinates (s,t) in [0,1] map to pixels (texels) in the image.

Loading and binding a texture

```
GLuint texID;
glGenTextures(1, &texID); // generate texture name
glBindTexture(GL_TEXTURE_2D, texID); // make it current

// Upload pixel data:
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height,
0, GL_RGB, GL_UNSIGNED_BYTE, pixels);

// Set filtering:
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

// Set wrapping:
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glEnable(GL_TEXTURE_2D);
```

glTexParameteri — filtering & wrapping

GL_NEAREST	Nearest-neighbour filtering. Fast, pixelated/blocky up close. Use for pixel-art style.
GL_LINEAR	Bilinear filtering. Smooth interpolation between texels. Better visual quality.
GL_LINEAR_MIPMAP_LINEAR	Trilinear filtering using mipmaps. Best quality for minified textures.
GL_REPEAT	Texture tiles infinitely. Coordinates wrap: 1.5 = 0.5.
GL_CLAMP_TO_EDGE	Edge texels are stretched. No tiling — good for decals and UI.
GL_MIRRORED_REPEAT	Texture tiles with alternating mirrors.

Assigning texture coordinates

```
glBindTexture(GL_TEXTURE_2D, texID);
glBegin(GL_QUADS);
glTexCoord2f(0.0, 0.0); glVertex3f(-1, -1, 0); // bottom-left
glTexCoord2f(1.0, 0.0); glVertex3f( 1, -1, 0); // bottom-right
glTexCoord2f(1.0, 1.0); glVertex3f( 1, 1, 0); // top-right
glTexCoord2f(0.0, 1.0); glVertex3f(-1, 1, 0); // top-left
glEnd();
```

Exam tip: Texture coordinates are (s,t) or (u,v), both in [0,1]. (0,0) is bottom-left in OpenGL. Values outside [0,1] depend on the GL_TEXTURE_WRAP setting.

glTexEnvf — texture environment

```
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
// GL_MODULATE : texture colour * lighting colour (default)
// GL_REPLACE  : texture colour replaces fragment colour entirely
// GL_DECAL    : blend texture over base colour using alpha
// GL_ADD       : add texture colour to fragment colour
```

Exam tip: GL_REPLACE ignores lighting on the textured surface — useful for UI elements or skymaps. GL_MODULATE is usually what you want for lit 3D objects.

9. Anti-Aliasing

Aliasing is the jagged "staircase" effect on diagonal edges caused by sampling a continuous signal at discrete pixel locations. Anti-aliasing reduces this by blending edge pixels.

Line and point smoothing

```
glEnable(GL_LINE_SMOOTH);
glEnable(GL_POINT_SMOOTH);
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glHint(GL_LINE_SMOOTH_HINT, GL_NICEST); // quality hint
glHint(GL_LINE_SMOOTH_HINT, GL_FASTEST); // speed hint
```

Exam tip: `GL_LINE_SMOOTH` requires blending to be enabled — the anti-aliasing works by writing partially transparent fragments at the edge. Without `GL_BLEND` enabled, it has no visible effect.

Multisampling (MSAA)

Multisample Anti-Aliasing renders each pixel at multiple sub-pixel sample points and averages the results. Configured at context creation time via GLUT.

```
// Request multisampled framebuffer when creating window:
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH |
GLUT_MULTISAMPLE);

// Enable during rendering:
glEnable(GL_MULTISAMPLE);

// Common sample counts: 2x, 4x, 8x, 16x
// More samples = better quality but more GPU memory and fill rate cost
```

2x MSAA	2 samples per pixel. Minimal performance cost, noticeable improvement.
4x MSAA	Most common setting. Good quality/performance balance.
8x MSAA	High quality. Diminishing returns vs 4x for most scenes.
SSAA	Supersampling — render at higher resolution then downsample. Highest quality, very expensive.

glHint — quality vs speed tradeoff


```
glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST); // better texture  
interpolation  
glHint(GL_FOG_HINT, GL_FASTEST); // per-vertex fog (faster)  
glHint(GL_POLYGON_SMOOTH_HINT, GL_NICEST); // smooth polygon edges
```

GL_NICEST	Driver chooses highest quality algorithm. May be slower.
GL_FASTEST	Driver chooses fastest algorithm. May reduce quality.
GL_DONT_CARE	Driver chooses freely — default behaviour.

Exam tip: `glHint()` is advisory only — the driver is not required to honour it. Different GPUs may produce identical results regardless of the hint.

10. Multimedia & Practical Patterns

Standard GLUT application skeleton

```
#include

float angle = 0;

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(0,2,5, 0,0,0, 0,1,0);
    glRotatef(angle, 0,1,0);
    glutSolidTeapot(1.0);
    glutSwapBuffers();
}

void reshape(int w, int h) {
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45, (float)w/h, 0.1, 100);
    glMatrixMode(GL_MODELVIEW);
}

void timer(int v) {
    angle += 1.0f;
    glutPostRedisplay();
    glutTimerFunc(16, timer, 0);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("My Scene");
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_LIGHTING); glEnable(GL_LIGHT0);
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutTimerFunc(16, timer, 0);
    glutMainLoop();
    return 0;
}
```

Common exam code patterns to recognise

Pattern / Bug	What it causes
Missing glLoadIdentity() before gluLookAt	Camera transforms accumulate — scene drifts each frame
glTranslate before glRotate	Object rotates around world origin, not its own centre
glRotate before glTranslate	Object rotates around its own centre, then moves
zNear set very small (0.001)	Z-fighting — surfaces at same depth flicker
Mismatched Push/PopMatrix	Subsequent objects drawn in wrong coordinate space
No GL_NORMALIZE with glScale	Normals distorted — lighting looks wrong after scaling
Blending without depth sort	Transparent objects render incorrectly — wrong layering
Wrong GL_TEXTURE_WRAP	Texture stretches at edges or tiles unexpectedly
glutDisplayFunc not called	Nothing is drawn — black window
GLUT_SINGLE instead of GLUT_DOUBLE	Flickering/tearing as scene is drawn directly to screen